

Contents

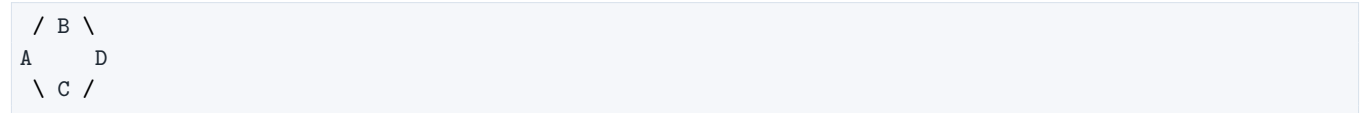
1 Vortex Scheduler: Centralized Scheduling for DAG-Structured Jobs	1
1.1 At A Glance	1
1.1.1 Messages	1
1.2 Worker Internals	2
1.2.1 A: Join	3
1.3 Scheduler Internals	5

1 Vortex Scheduler: Centralized Scheduling for DAG-Structured Jobs

Vortex Scheduler is a centralized scheduling architecture for executing DAG-structured jobs across a pool of workers in a multi-node system. The primary objective is to maximize **job goodput** with respect to a configurable service-level-objective (SLO). Vortex Scheduler is built on top of cascade, an AI/ML application hosting framework powered by optimized RDMA datapaths (derecho).

In this system, a **job** is composed of multiple dependent **tasks**. Tasks may only execute when their upstream dependencies have been satisfied. Jobs are submitted to a single scheduling node, which is responsible for making global placement and dispatch decisions across all workers.

Consider the canonical diamond-shaped DAG below:



For a job J_1 with tasks $\{A_1, B_1, C_1, D_1\}$:

- A_1 must complete before B_1 and C_1 become runnable.
- D_1 may execute only after both B_1 and C_1 complete.
- J_1 contributes to goodput only if all four tasks complete and the job satisfies the target SLO.

1.1 At A Glance

VortexScheduler consists of a single **scheduling** node and multiple **worker** nodes. The scheduling node maintains a global view of job state and dispatches work across the worker pool. Each worker node is provisioned with a fixed subset of task-processing capabilities and is responsible for executing only a subset of the task types that may appear in a job's DAG. For example, a worker node N_A may expose two task processors: one capable of executing tasks of type T_A , and one capable of executing tasks of type T_B .

Scheduler-facing placement decisions are expressed in terms of a logical `worker_id`, rather than raw Cascade subgroup and shard coordinates. Each node maintains a local **Worker Placement Table** that maps `worker_id` values to the corresponding Cascade destination metadata, including the object type, subgroup index, and shard index required to deliver a message to that worker.

A key design decision in VortexScheduler is that, for worker-facing message-passing paths, **replication per shard is fixed to 1**. Under this constraint, each (`subgroup_index`, `shard_index`) pair identifies exactly one destination node. This allows the scheduler to reason in terms of workers, while the runtime resolves each `worker_id` into a unique Cascade address at dispatch time.

1.1.1 Messages

VortexScheduler defines two message classes: **Control Messages** and **Data Messages**. Control messages communicate scheduler intent and worker-local execution state. Data messages carry task outputs and intermediate values between dependent stages of a job DAG.

VortexScheduler adopts Cascade's key-value-store execution model. User-defined logic (UDL) publishes values to named paths in the key-value store, and updates on those paths trigger the UDL handlers registered for the corresponding key

or path prefix.

```

struct TaskRef {
    job_id      : u16      // unique to the job
    task_id     : u16      // identifies the component
}

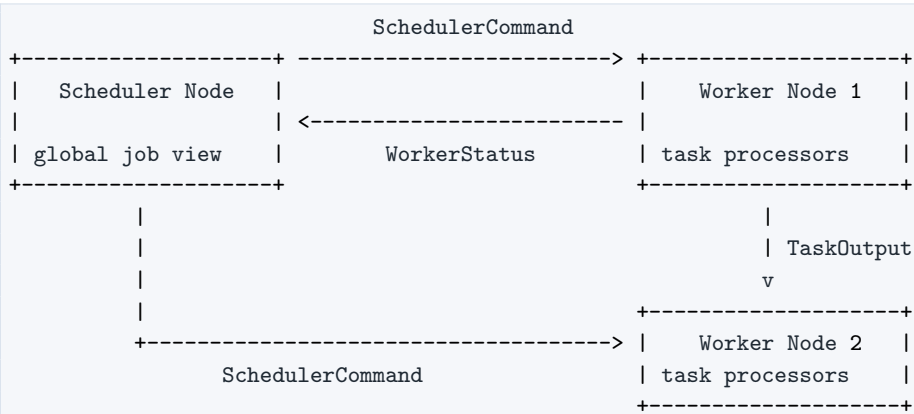
struct Decision {
    worker_id   : u16
    target_task_id : u16
    task        : TaskRef
}

// sent from scheduler to worker nodes
message SchedulerCommand{
    decisions    : vec<Decision>
    cancellations : vec<TaskRef> // task instances to drop
}

// sent from worker nodes to the central scheduler
message WorkerStatus {
    worker_id    : u16
    task_id      : u16
    queue_size   : u16      // number of tasks in the local queue
    time_to_empty : u64      // expected time until no tasks left
    completed    : vec<TaskRef> // completed task instances
}

// sent between local nodes
// the previous task is encoded by the path the value was received on
message TaskOutput {
    worker_id    : u16
    job_id      : u16
    num_targets  : u16      // < K
    target_task_ids : [u16; K]
    data        : binary blob // user defined type (serde responsibility is booted to the user)
}

```



1.2 Worker Internals

The worker runtime is organized around Cascade’s callback model: incoming values are delivered to `ocdpo_handler` on a Cascade-managed thread, and the lifetime of the received blob ends when the callback returns. As a result, the worker must treat `ocdpo_handler` strictly as an **ingress boundary**. Any payload that must outlive the callback must

be copied or moved into worker-owned memory before the callback completes.

For this reason, the worker should be structured as a pipeline of internal services rather than performing task execution directly inside the callback. We reduce it to four main stages: **join**, **batch**, **execute**, and **scatter**.

1. Join: **DAG Registry, Ingress Binding Table, Task Join Service**
2. Batch + Execute: Accomplished through the **python-udl interface**
3. Scatter:

1.2.1 A: Join

OCDPO thread: The Join layer is the first ownership boundary. Its responsibility is to deserialize enough metadata to identify the incoming message type, extend the lifetime of the object via a memcpy, and enqueue an internal representation for the execution state. For **TaskOutput** messages, this means extracting the **TaskRef** and a copy of the binary payload, then handing it off to the **Task Join Service**.

Ingress thread: Located within the **TaskJoinService** whose responsibility is to act as a temporary holding arena which emits computable tasks (all dependencies are present). It contains four internal pieces:

1. **Input Arena** - implemented as a size-classed segmented holding arena. The arena owns copied payload bytes and returns a stable **BlobHandle**. Small, medium, and large payloads may use different segment pools, but all are referenced through the same handle type.
2. **Ingress Queue** - a single-producer / single-consumer queue from the OCDPO thread to the join thread. The queue carries only lightweight metadata and **BlobHandles**; the payload bytes remain in the input arena.
3. **Join Table** - keyed by destination **TaskRef**. It tracks which dependency slots have been satisfied and emits a **TaskBinding** when all upstream components are present.
4. **DAG Registry** - join logic cannot rely on **task_id** alone since different jobs may have different DAG shapes. Thus, each job must resolve to a graph identifier before the join can be evaluated. For each (**graph_id**, **task_id**) pair, the registry must provide:
 - **expected_inputs** - number of required input slots
 - **invoke_handler** - worker-local task processor that executes the task
 - **downstream** - output consumers

The notification mechanism is intentionally edge-triggered and minimal:

1. the OCDPO thread pushes an **IngressRecord** into the ingress queue
2. if the queue transitioned from empty to non-empty, the OCDPO thread performs a non-blocking wakeup of the join thread
3. the join thread drains the queue until empty before sleeping again

This avoids one wakeup per message during bursts. A burst of **TaskOutput** arrivals should therefore cause one queue transition, one wakeup, and a full queue drain by the join thread.

```
struct BlobHandle {
    pool_class    : u8
    segment_id    : u32
    offset        : u32
    size          : u32
}

struct TaskBinding {
    task          : TaskRef
    processor_id  : u16
    inputs        : vec<BlobHandle>
}

struct TaskOutputHeader {
```

```

worker_id      : u16
job_id         : u16
num_targets    : u16           // < K
target_task_ids : [u16; K]
}

struct IngressRecord {
    header      : TaskOutputHeader
    dependency_slot : u16
    payload      : BlobHandle
}

struct TaskSpec {
    expected_inputs : u16
    processor_id    : u16
    supports_batching: bool
}

```

```

// example workflow

// inside taskJoinService::recv
void recv(const std::string& string_key, const Blob& blob) {
    const TaskOutputHeader header = decode_task_output_header(blob);
    const PayloadSlice payload = decode_payload(blob);           // lightweight slice type (no-
                                                                    memcpy yet)
    const uint16_t dep_slot = ingress_bindings.lookup(string_key);

    // divided into small (< 4Kb ), medium (< 1 MB), large pools ( >= 1 MB)
    // all pools use a segmented bump allocator

    // e.g. a segment will be allocated 500 MB at a time, and allocation happens in monotonous fashion
    // after a segment's data is consumed, it can be recycled.

    // memcpy here
    BlobHandle handle = holding_arena.extend(payload, ref_cnt=header.num_targets);

    const bool was_empty = received.was_empty_before_push();
    received.push(IngressRecord{
        .header      = header,
        .dependency_slot = dep_slot,
        .payload      = handle,
    });

    // notify ingress thread only on empty -> non-empty transition
    if (was_empty) {
        ingress_notifier.notify_one();
    }
}

// ocdpo handler
void ExampleUdl::ocdpo_handler(const node_id_t sender,
                                const std::string& object_pool_pathname,
                                const std::string& key_string,
                                const ObjectWithStringKey& object,
                                const emit_func_t& emit,
                                DefaultCascadeContextType* typed_ctxt,
                                uint32_t worker_id) {

```

```

    if (is_data_key(key_string)) {
        // extends lifetime via memcpy
        // information lies in a bump allocator
        task_join_service.recv(key_string, object.blob);
    }
}

// ingress thread
void loop() {
    while (true) {
        ingress_notifier.wait();

        while (received.has_items()) {
            const IngressRecord rec = received.front();
            const uint32_t graph_id = job_context.lookup(rec.header.job_id).graph_id;

            for (int i = 0; i < rec.header.num_targets; i++) {
                const uint16_t target_task_id = rec.header.target_task_ids[i];
                const TaskSpec& spec = dag_registry.lookup(graph_id, target_task_id);

                auto ready = join_table.put(
                    TaskRef{rec.header.job_id, target_task_id},
                    graph_id,
                    target_task_id,
                    rec.dependency_slot,
                    rec.payload,
                    spec
                );

                if (ready.has_value()) {
                    processor_queues[spec.processor_id].push(std::move(*ready));
                }
            }

            // drops the recv-local reference; remaining references are held by the join table
            holding_arena.release(rec.payload);
            received.pop();
        }
    }
}

```

1.3 Scheduler Internals